

The AI Prompt Cookbook

Keep this. It is the part of the course that still works six months from now, on a different dataset, for a question nobody has asked you yet.

You are not learning to write Earth Engine code. You are learning to **specify** what you want precisely enough that a machine can write it, and to **check** the answer well enough that you would put your name on it. The second half is the hard half.

1 The PROMPT recipe

Six slots. Fill all six and the code usually works first time. Skip one and you will spend ten minutes fixing what a single sentence would have prevented.

SLOT	WHAT IT ANSWERS	EXAMPLE
Purpose	What map or number do you want?	"Calculate how many hectares of each land cover class are in my province."
Resource	Which dataset, by exact ID?	ESA/WorldCover/v200/2021 , band Map
Outline	Which area, which dates?	"Region is the variable <code>aoi</code> . Dates 2024-11-01 to 2025-02-28."
Must-haves	Scale, units, palette, legend	"Use scale=100. Give me hectares, not square metres. Add a legend."
Platform	Where will this run?	"Google Colab, <code>ee</code> and <code>geemap</code> imported and authenticated."
Test	How will you know it worked?	"Print the total so I can check it against the province area."

THE SLOT EVERYONE SKIPS

Test. Without it the AI gives you a beautiful map and no way to tell whether it is right. Always ask for one printed number you can check against something you already know.

2 The error loop

Your code will break. That is normal and it is not a sign you are bad at this. What separates people who get unstuck quickly from people who get stuck for an hour is entirely mechanical:

DO THIS	NOT THIS
STOP	Do not change three things at once and re-run hoping.
COPY the whole error	Not just the last line. The traceback above it says <i>which cell and which line</i> .
PASTE it back with one sentence of context	Not "it doesn't work". Say what you were trying to do.
ASK for corrected code	Not "why did this happen?" — you want the fix, not an essay.
Re-run and re-check	Do not assume the fix worked because the error disappeared.

This is the error I got. Please give me the corrected code.

I was trying to: `calculate hectares of forest in my province`

`[paste the ENTIRE red error message here, all of it]`

NEVER RETYPE CODE BY HAND

Copy and paste. Every character you retype is a chance to introduce a typo that produces a *different* error, and now you are debugging two problems instead of one.

3 The sniff test

An AI will confidently hand you code that runs perfectly and computes the wrong thing. Running without error is not evidence of correctness. Before you believe any number, check it against something you already know.

QUANTITY	PLAUSIBLE RANGE	IF YOU SEE THIS, SOMETHING IS WRONG
NDVI	-1 to 1; tropical forest 0.7–0.9	3000 → forgot to divide by 10000. All zeros → the mask removed everything.
Area of a class	less than the area of your province	More forest than land → you summed square metres and called them hectares.
Aboveground biomass	100–400 Mg/ha in tropical forest	5 → you are averaging over water. 20,000 → wrong units.
Carbon density	40–200 Mg C/ha	Half what you expected → you applied *0.47 to a dataset that was already carbon.
Temperature from ERA5	15–35 °C	Around 300 → that is Kelvin. Subtract 273.15.
Classification accuracy	0.70–0.92	Above 0.98 → your test points leaked into training.
Burned area	a few % of a province	Half the province → you used dNBR without masking to a real burn scar.

THE ONE QUESTION TO ASK EVERY SINGLE TIME

"Is this number biomass or carbon? Hectares or square metres? Celsius or Kelvin?" Almost every wrong answer in this field is a units error wearing a convincing map as a disguise.

4 Ready-made prompt cards

Each card is a filled-in PROMPT recipe. Replace the highlighted parts. These are the exact prompts used in the labs, and they have all been tested.

CARD 01 Cloud-free satellite image of my area

You are a Google Earth Engine Python expert.

PURPOSE

Build one cloud-free Sentinel-2 image of my study area.

RESOURCE

Use dataset COPERNICUS/S2_SR_HARMONIZED.

OUTLINE

Area is in the variable `aoi`. Dates 2024-11-01 to 2025-02-28.

MUST

Mask cloud and shadow with the SCL band (drop values 3, 8, 9, 10).
Divide by 10000 for reflectance. Take the median.
Keep bands B2, B3, B4, B8, B11, B12.

PLATFORM

Google Colab, `ee` and `geemap` imported and authenticated.

TEST

Print how many scenes went into the composite.

Good output looks like:

a number between about 50 and 400 scenes, and a map where clouds are gone but the image still looks like a real landscape.

IF IT GOES WRONG	SAY THIS NEXT
Zero scenes returned	"The collection is empty. Please loosen the cloud filter and print the date range and bounds you are actually using."
Image is black or white	"The display is saturated. The values are 0-1 reflectance – please set min 0 and max 0.3 for true colour."
Still full of cloud	"Clouds remain. Please mask using the SCL band values 3, 8, 9 and 10, and also filter scenes by CLOUDY_PIXEL_PERCENTAGE below 30."

CARD 02 Area of each land cover class, in hectares

PURPOSE Calculate how many hectares of each land cover class are in my area.
RESOURCE Use [ESA/WorldCover/v200/2021](#), band [Map](#).
OUTLINE Region is `aoi`.
MUST Use `ee.Image.pixelArea()` divided by 10000 to get hectares.
Group by the class band with a grouped sum reducer.
Use `scale=100` and `maxPixels=1e9`.
Print a table sorted largest first, with a percentage column.
PLATFORM Google Colab.
TEST The class areas must add up to roughly my region's total area.

Good output looks like: a table whose total is within a few percent of the area you printed when you defined your AOI.

IF IT GOES WRONG

SAY THIS NEXT

Numbers are 10,000× too big

"These look like square metres. Please divide pixel area by 10000 to get hectares."

"Too many pixels" error

"Please add `maxPixels=1e9` and `bestEffort=True`, and raise `scale` to 100."

Only one class returned

"I expected several classes. Please confirm you are grouping on the class band and not on the area band."

CARD 03 Classify forest types with Random Forest

PURPOSE Train a Random Forest to classify forest types and tell me how accurate it really is.
RESOURCE Training points are in `samples` (property "class"), predictors in `FEATURES`.
OUTLINE Split 70% train / 30% test using `randomColumn` with seed 42.
MUST Use `ee.Classifier.smileRandomForest` with 50 trees.
Compute the confusion matrix on the TEST set only, never the training set.
Print overall accuracy and kappa.
PLATFORM Google Colab, Earth Engine Python API.
TEST Print the confusion matrix so I can see which classes get confused.

Good output looks like: accuracy 0.75–0.92, and a confusion matrix where the mistakes are between classes that genuinely look alike.

IF IT GOES WRONG

SAY THIS NEXT

Accuracy is 0.99+

"This looks like the test set leaked into training. Please confirm the confusion matrix uses only the held-out 30%."

One class is never predicted

"Class N is never predicted. Please check how many training points exist for it and use `stratifiedSample` to balance the classes."

Classifier training fails

"Please confirm every property in `FEATURES` exists as a band on the image I am classifying."

CARD 04 Total carbon stock and CO₂ equivalent

PURPOSE	Calculate the TOTAL carbon stored in the forest of my area, and its CO ₂ equivalent.
RESOURCE	Use NASA/ORNL/biomass_carbon_density/v1. This is an ImageCollection with one image – load it with <code>.first()</code> , not <code>ee.Image()</code> . Bands <code>agb</code> and <code>bgb</code> are ALREADY in Mg C/ha. Do NOT apply a 0.47 carbon fraction.
OUTLINE	Restrict to forest pixels. Region is <code>`aoi`</code> . Use <code>scale=300</code> .
MUST	Multiply carbon density by pixel area in hectares BEFORE summing. Convert carbon to CO ₂ e by multiplying by 44/12.
PLATFORM	Google Colab, Earth Engine Python API.
TEST	Print total tonnes of carbon and total tonnes of CO ₂ e with thousands separators.

Good output looks like: tens of millions of tonnes of carbon for a forested province.

IF IT GOES WRONG

SAY THIS NEXT

Asset ... is not an Image	"That asset is an ImageCollection with a single image. Please load it with <code>ee.ImageCollection(...).first()</code> ."
Answer looks half what you expected	"You applied a 0.47 carbon fraction. This dataset is already carbon – please remove that step."
Answer is in the thousands only	"You averaged instead of summing. Please multiply density by pixel area in hectares and then sum."

CARD 05 How many hectares burned

PURPOSE	Find how many hectares burned in my area in each of the last three fire seasons.
RESOURCE	Use MODIS/061/MCD64A1, band <code>BurnDate</code> (0 = unburned, 1–366 = day of year).
OUTLINE	Region is <code>`aoi`</code> . For each year <code>2023, 2024, 2025</code> , dates January 1 to June 1.
MUST	Take the max of <code>BurnDate</code> over the season, treat greater than 0 as burned, multiply by pixel area in hectares, and sum. Use <code>scale=500</code> .
PLATFORM	Google Colab, Earth Engine Python API.
TEST	Print one line per year so I can compare seasons.

Good output looks like: a few percent of your province per year, varying noticeably between years.

IF IT GOES WRONG

SAY THIS NEXT

Every year identical	"The three years give the same number. Please confirm the date filter is inside the loop and actually changing."
You used <code>dNBR</code> and got a huge number	" <code>dNBR</code> is picking up deciduous leaf-off, not fire. Please mask the <code>dNBR</code> result to pixels that <code>MCD64A1</code> confirms burned."

CARD 06 Monthly rainfall and temperature chart

PURPOSE Chart monthly rainfall and temperature for one year in my area.
RESOURCE Rainfall: UCSB-CHG/CHIRPS/DAILY, band precipitation, mm per day.
Temperature: ECMWF/ERA5_LAND/MONTHLY_AGGR, band temperature_2m, in KELVIN.
OUTLINE Region is `aoi`. Year **2024**.
MUST Sum daily rainfall to a monthly total, then take the spatial mean (scale=5566).
Convert temperature from Kelvin to Celsius by subtracting 273.15 (scale=11132).
Plot rainfall as bars on the left axis and temperature as a line on the right axis.
PLATFORM Google Colab with matplotlib.
TEST Print both lists of 12 numbers before plotting.

IF IT GOES WRONG

SAY THIS NEXT

Temperatures around 300

"That is Kelvin. Please subtract 273.15."

Rainfall looks far too small

"You took the mean of daily values. Please sum the daily images across the month first, then take the spatial mean."

5 How to catch a hallucination

AI assistants invent dataset IDs, band names and function names that look entirely plausible. They do it most often when the real thing is obscure. Three defences, in order of speed:

1. **Check the ID against the cheat sheet** before you run anything. This catches most of it in five seconds.
2. **Print the band names** rather than trusting them: `print(img.bandNames().getInfo())`
3. **Search the official catalog** for the ID. If a Google page for it does not exist, the dataset does not exist.

REAL EXAMPLES FROM THIS COURSE

- UMD/hansen/global_forest_change_2024_v1_12 — **real, but superseded**. The current version is 2025_v1_13. Assistants trained on older material still suggest the old one.
- `ee.Image('NASA/ORNL/biomass_carbon_density/v1')` — **right ID, wrong type**. It is an ImageCollection. Every assistant we tested got this wrong.
- Any band called NDVI on a raw Sentinel-2 image — **does not exist**. You have to compute it.

6 Working with a free AI account

USING ANTIGRAVITY FOR THIS COURSE

Antigravity is the AI assistant this course is taught with — free, no credit card. It is also a full agentic IDE that can open and run files on its own. **Do not use that here**. Use only its chat panel: paste your prompt, read the code it gives back, then copy that code into a **Colab** cell and run it there. Every lab in this course was built and verified in Colab — that is where your maps will actually display, and it is also where your Earth Engine authentication already lives.

- **Keep one conversation per lab**. The assistant remembers what `aoi` is; starting a fresh chat for every question throws that away and wastes your message quota.
- **Say what already exists**. *"The variable `composite` already holds my image"* stops it from regenerating fifty lines you already ran.

- **If AntigraVity hits its weekly rate limit**, switch to Gemini inside Colab — it is free, unlimited in practice, and already knows what is in your notebook.
- **Ask for short code.** Adding *"keep it under 25 lines and comment each step"* gives you something you can actually read and debug.

THE HABIT WORTH TAKING AWAY

The AI is fast and confident and does not know whether its answer is true. You are slower, but you are the only one in the loop who can tell. **Specify carefully, then check the number.** That is the whole job.